AUTOMATION · HOME CLUSTER · K3S

# Renovate
# automated dependency updates for the home cluster

Every pinned image tag in this repo is a dependency that silently ages. Renovate is the bot that notices: a **self-hosted** instance pointed at the Forgejo copy of the repo (`brian/home-lab` on `https://forgejo.lan`) that scans every manifest, checks upstream registries, and opens PRs when something is behind. Merge a PR and — for the services already under Argo CD — the cluster updates itself. This doc explains what's deployed, how the tracking mechanics actually work, one worked example, and where the blind spots are.

**Runs** daily · 06:00 America/New_York | **Target** brian/home-lab @ forgejo.lan | **Image** renovate/renovate:43.251.3

**Status** deployed — first run pending scoped token

| 1/day | 3 | 5/hr | 4 |
|---|---|---|---|
| BATCH RUN | MANAGERS | PR BUDGET | AUTO-DEPLOY |
| CronJob wakes at 06:00, scans, opens PRs, exits | kubernetes + kustomize + dockerfile parse the repo | prHourlyLimit 5, prConcurrentLimit 10 — no PR flood | Argo CD starter tranche syncs merged PRs hands-free |

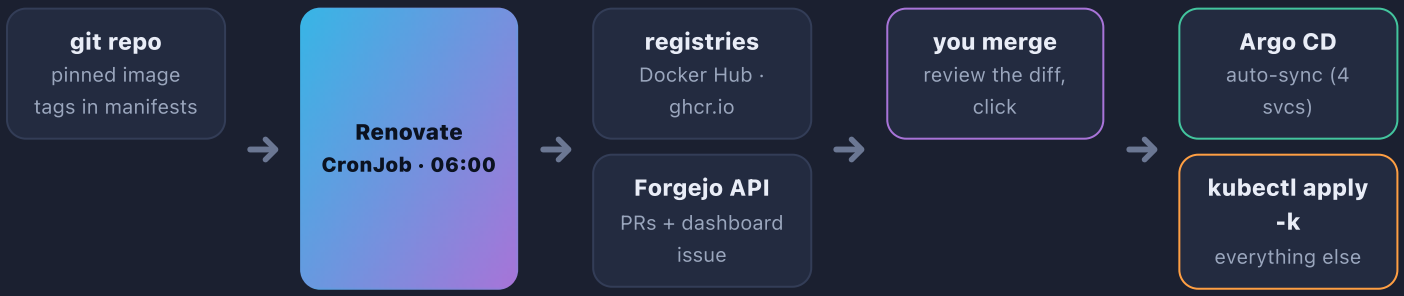
## Get the mental model right first

**Unlearn this: "Renovate is a service you run"**

There is no Renovate server, no webhook listener, no long-running pod. Renovate is a **batch job**: wake up, clone the repo, parse it, query registries, open/update PRs on Forgejo, exit. Between runs it consumes **zero** resources and holds **zero** state — everything it "remembers" (open PRs, the dashboard issue, branch state) lives *in Forgejo itself*. That's why a CronJob is the whole deployment, and why deleting the namespace loses nothing.

**Why self-hosted?** The hosted Renovate GitHub App runs in Mend's cloud — it can reach `github.com`, not a Forgejo that only exists on this LAN behind a mkcert certificate. Since the repo of record for the cluster is the Forgejo copy (the GitOps source Argo CD watches, home-lab#29), the bot has to live where the repo lives: **inside the cluster**, trusting the LAN CA, talking to `forgejo.lan` over the Gitea-compatible API.

## The pipeline, end to end




The human stays exactly at the merge button. Everything left of it is automated noticing; everything right of it is deployment — automated only for the services migrated to GitOps so far.

DEPLOYMENT · CLUSTERS/HOME/RENOVATE

## The whole deployment is one CronJob

`clusters/home/renovate/` is two manifests: a namespace and a CronJob. The interesting decisions, straight from `cronjob.yaml`:

| FIELD | VALUE | WHY |
|---|---|---|
| **image** | `renovate/renovate:43.251.3` | Pinned — and Renovate tracks **its own tag**, so it self-updates via its own PRs |
| **schedule** | `"0 6 * * *"` · `America/New_York` | One morning sweep; PRs are waiting with coffee |
| **concurrencyPolicy** | `Forbid` | Two Renovates racing on one repo = duplicate branches |
| **limits** | `activeDeadlineSeconds: 3600` · `backoffLimit: 1` | A hung run dies in an hour; one retry, then wait for tomorrow |
| **resources** | 250m CPU / 512Mi req · 2Gi mem limit | Node's a big clone + parse; bounded so it can't squeeze GPU workloads |

### Platform wiring (env)

```
RENOVATE_PLATFORM=gitea        # Forgejo speaks
                               # the Gitea API
RENOVATE_ENDPOINT=https://forgejo.lan/api/v1
RENOVATE_REPOSITORIES=brian/home-lab
RENOVATE_ONBOARDING=false    # renovate.json is
                               # committed; skip the
                               # onboarding PR dance
```

### TLS trust for forgejo.lan

```
NODE_EXTRA_CA_CERTS=/etc/mkcert/rootCA.pem
GIT_SSL_CAINFO=/etc/mkcert/rootCA.pem
# mounted from the mkcert-ca ConfigMap
# (maintained by scripts/lan-certs.sh).
# Both needed: the Node.js process AND
# the git binary it shells out to must
# trust the LAN CA independently.
```

## The token — out-of-band, via Vaultwarden

Renovate authenticates to Forgejo with an API token from the k8s Secret `renovate-secrets` (key `token`) — never committed. Canonical copy lives in Vaultwarden item `renovate-forgejo`. Scopes (generated in Forgejo → Settings → Applications): `read:user` `write:repository` `read:issue` `write:issue` — enough to push branches, open PRs, and maintain the dashboard issue; nothing admin.

```
# 1 · stash the token in Vaultwarden (nothing touches disk):
printf %s '<token>' | scripts/vault-put.sh renovate-forgejo

# 2 · (re)create the in-cluster secret from the vault:
kubectl -n renovate delete secret renovate-secrets --ignore-not-found
kubectl -n renovate create secret generic renovate-secrets \
  --from-literal=token="$(scripts/vault-secret.sh renovate-forgejo)"
```

### Status caveat — deployed, not yet proven

As of this writing the properly-scoped token is **still pending** — the CronJob, config, and TLS trust are all in place, but **the first successful run hasn't happened yet**. Until the bootstrap above lands, expect the 06:00 job to fail authentication. First real run: see the expectations on p.5.

### Renovate manages itself — under GitOps

`clusters/home/renovate` is in the Argo CD starter tranche (`clusters/home/argocd/apps/home-services.yaml`). So the loop closes: Renovate opens a PR bumping `renovate/renovate:43.251.3` → merge → Argo rolls the CronJob to the new tag. The updater gets updated through its own PRs.

## Repo-side policy — `renovate.json` at the repo root

The CronJob is only the scheduler; *behavior* is committed to the repo it manages, so policy changes are just commits:

```
{
  "extends": ["config:recommended", ":dependencyDashboard"],
  "timezone": "America/New_York",
  "prHourlyLimit": 5, "prConcurrentLimit": 10,     # tame the first-run burst
  "kubernetes": { "managerFilePatterns":
    ["/^clusters\\/.+\\.ya?ml$/", "/^services\\/.+\\.ya?ml$/"] }, # scope the k8s manager
  "packageRules": [
    { # all linuxserver.io images travel together in ONE PR
      "matchPackageNames": ["linuxserver/**", "lscr.io/linuxserver/**"],
      "groupName": "linuxserver images" },
    { # majors get an extra label so they stand out in the PR list
      "matchUpdateTypes": ["major"], "labels": ["renovate", "major"] } ]
}
```

## How tracking actually works — four steps per run

**1 Managers parse files into dependencies**

A *manager* is a file-format parser. Three do the work here — see the table below. Each match becomes a (package, current-version, registry) triple. No agents on nodes, no cluster API calls — **the git repo is the only input**.

**2 Datasources query the registries**

Each dependency is looked up against its registry — Docker Hub for library images, `ghcr.io` for GitHub-published ones — listing available tags via the standard registry API.

**3 Versioning logic sorts what "newer" means**

Loose-semver comparison that also handles CalVer tags like `v2026.6.19` (p.4) and classifies each jump as patch / minor / major — which is what the grouping and labeling rules key off.

**4 One branch + PR per update (unless grouped)**

Renovate pushes a branch editing **every occurrence** of the old tag, opens a PR with release notes when it can find them, and keeps the PR rebased as `main` moves. Close a PR without merging and that version is remembered as ignored.

| MANAGER | PARSES | WHERE IT BITES IN THIS REPO |
|---|---|---|
| `kubernetes` | `image:` fields in raw manifests | `clusters/**/*.yaml` + `services/**/*.yaml` (scoped above — the biggest surface) |
| `kustomize` | `images:` overrides + `helmCharts:` versions | every `kustomization.yaml` — chart bumps become PRs too |
| `dockerfile` | `FROM` lines | e.g. `clusters/home/hermes/image/Dockerfile` |

**The Dependency Dashboard is the audit view.** `:dependencyDashboard` makes Renovate maintain a single pinned issue in Forgejo listing **every dependency it detected and every update pending, open, or ignored**. That issue is the one-glance answer to "is the cluster's git state up to date?" — and its checkboxes let you force-retry or un-ignore an update without touching config.

### Merging a PR *is* the update — for some services

The Argo CD starter tranche — **mailpit, qbittorrent, rampart, and renovate itself** (`clusters/home/argocd/apps/home-services.yaml`) — auto-syncs on merge (`selfHeal` on, `prune` off: drift reverts, deletions stay manual). Everything else still needs a `kubectl apply -k clusters/home/<svc>` after merging. Renovate makes the gap visible: every merged-but-not-applied PR is drift you chose.

## Worked example — the hermes image bump

`clusters/home/hermes/deployment.yaml` pins `nousresearch/hermes-agent:v2026.6.19` — a **CalVer** tag (year.month.day). Upstream's latest is `v2026.7.1`. On its next run, Renovate will open **one PR** that touches **five occurrences**:

| FILE | OCCURRENCES | FOUND BY |
|---|---|---|
| `clusters/home/hermes/deployment.yaml` | **4** — main container + three initContainers (bootstrap steps reuse the image) | `kubernetes` manager |
| `clusters/home/hermes/image/Dockerfile` | **1** — the `FROM nousresearch/hermes-agent:v2026.6.19` base of the -otel derivative | `dockerfile` manager |

All five flip to `v2026.7.1` atomically in one branch — no "initContainer on the old tag" skew, which is exactly the mistake a hand edit makes.

**Subtlety: image tags are tracked, software releases are not**

Hermes-the-software versions like `v0.17.0 → v0.18.0` appear **nowhere** in this repo — only the **image tag** does. Renovate compares registry tags, full stop. If upstream shipped v0.18.0 without pushing a new image tag, Renovate would (correctly, by its rules) stay silent. The bot audits *what the repo pins*, not upstream changelogs.

**Follow-on chore the bot can't do**

The derived `harbor.lan/library/hermes-agent:*-otel` image is built *from* that Dockerfile locally. After merging a base bump, the -otel image must be **manually rebuilt and pushed** — Renovate updated the recipe, not the cake.

## Blind spots — what Renovate does not see

| BLIND SPOT | EXAMPLES IN THIS REPO | WHY · FIX |
|---|---|---|
| **Script-downloaded binaries** `fixable` | kubectl `v1.35.5` + Bitwarden CLI `2026.6.0` URLs inside Hermes initContainers; versions baked into `scripts/*.sh` | Versions live in shell strings no manager parses. Fix later with **custom regex managers** — teach Renovate the URL patterns. |
| **Unpinned tags** `unfixable as-is` | `latest`, `main`, `latest-spark-arm64` | Nothing to compare — a floating tag has no version. The real fix is pinning; until then these update invisibly on every pull. |
| **Locally-built images** `out of scope` | `harbor.lan/library/rampart:v0.1.3`, the hermes `-otel` derivative | No upstream registry to poll — *you* are upstream. (Renovate also has no harbor.lan credentials or CA config anyway.) |
| **Runtime drift** `different tool` | a running pod whose image differs from git | Renovate audits **git only**. Runtime-vs-latest is jetstack `version-checker`'s job — a possible future addition feeding the Grafana stack. |

Honest summary: Renovate covers the pinned-image surface — the majority of what this repo declares — and is structurally blind to everything above. Knowing the boundary is the point of this table.

## Day-2 operations

```
# trigger an off-schedule run (don't wait for 06:00):
kubectl create job --from=cronjob/renovate renovate-manual -n renovate
# watch what it did:
kubectl -n renovate logs job/<name>
# dry-run — full analysis, logs everything, changes NOTHING on Forgejo:
# add to the job's env:  - { name: RENOVATE_DRY_RUN, value: "full" }
```

### First-run expectations — don't panic at the PR burst

The first successful run against a repo this size produces a **burst of PRs** — every stale pin found at once, rate-limited to **5/hour** (so the backlog drains across runs, up to 10 open at a time) — plus the **Dependency Dashboard issue** appearing in Forgejo. PRs land authored by the token's user. Triage: merge the boring patch bumps, read the `major` -labeled ones.

## Quick reference

| THING | WHERE / VALUE |
|---|---|
| CronJob | `renovate/renovate` in ns `renovate` · `clusters/home/renovate/cronjob.yaml` |
| Schedule | `0 6 * * *` America/New_York · `concurrencyPolicy: Forbid` · 1h deadline |
| Platform / endpoint | `gitea` · `https://forgejo.lan/api/v1` · repo `brian/home-lab` |
| Policy file | `renovate.json` @ repo root — dashboard on, 5 PR/hr, 10 concurrent, linuxserver grouped, majors labeled |
| Token | Secret `renovate-secrets/token` ← Vaultwarden `renovate-forgejo` · scopes read:user, write:repository, read:issue, write:issue |
| TLS trust | `mkcert-ca` ConfigMap → `/etc/mkcert/rootCA.pem` via `NODE_EXTRA_CA_CERTS` + `GIT_SSL_CAINFO` |
| Auto-deploy on merge | mailpit · qbittorrent · rampart · renovate (Argo CD, selfHeal on / prune off) — rest: `kubectl apply -k` |
| Audit view | Dependency Dashboard issue in Forgejo — every tracked dep + pending update |

## What's next (in order)

1. **Finish the token bootstrap**
   Generate the scoped token in Forgejo → Settings → Applications, `vault-put.sh renovate-forgejo`, recreate the secret (p.2), then a manual run to watch the first burst live.
2. **Widen the Argo CD tranche**
   Every service migrated to GitOps turns a Renovate merge into a hands-free deploy.
3. **Close the blind spots (p.4)**
   Custom regex managers for the kubectl / Bitwarden CLI download URLs; later, jetstack `version-checker` for the runtime-drift half of the audit, graphed in Grafana.

### Bottom line

One pinned CronJob + one committed policy file, and the repo stops rotting silently: "are we behind?" becomes a pinned Forgejo issue, every update a one-click PR — for the GitOps tranche, also the deploy. It sees exactly what git pins, nothing more (p.4). Pending: the scoped token — a loaded pipeline that hasn't fired yet.